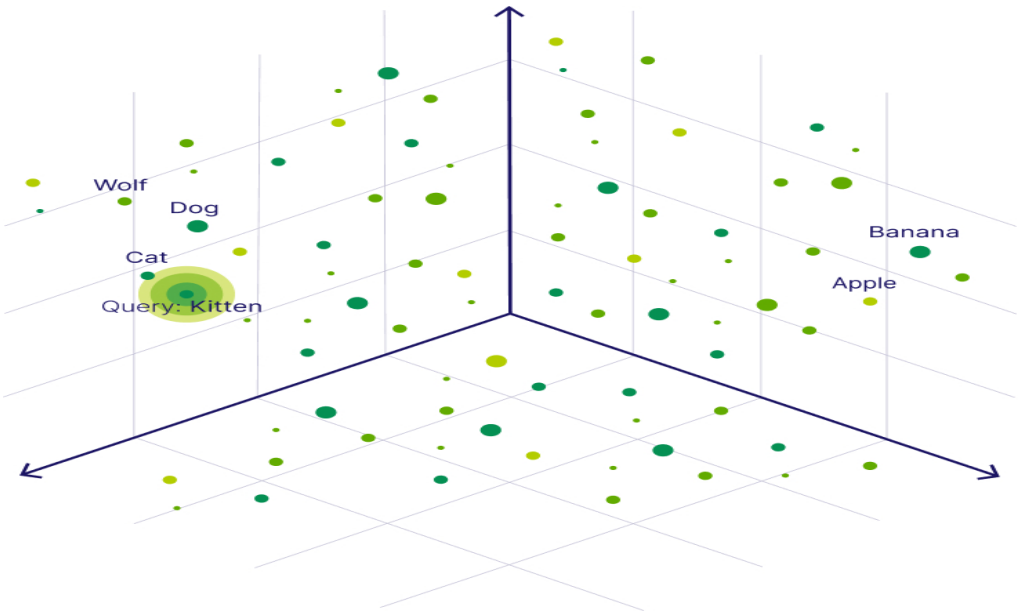
AN INTRO TO VECTOR DB

- Whenever you provide any prompt to any LLM model like ChatGPT, it converts your prompt which is in Natural Language into Vectors(embeddings) and this embeddings goes inside a Neural Network as an Input and then this neural network provides you the output.
- Some important terms:
    - **Parameters: The internal settings (weights and biases) that an AI learns during training. Think of them as millions or billions of adjustable knobs that fine-tune how the AI processes information**
    - **Billion Parameters (e.g., 7B, 70B): The scale of the model. A "7B" model has 7 billion knobs. Generally, more parameters mean a smarter model that understands more complex concepts, but it also requires more computer power to run.**
    - **Tokens: The basic building blocks of text for an AI. Models do not read whole words; they chop text into pieces (usually syllables or word fragments). One token is roughly equal to 4 characters or 0.75 words.**
    - **128k Tokens: means the AI has a memory capacity of roughly 96,000 words (about 300 to 400 pages of text) for a single conversation.**
        - 1 token is approximately equal to ≈ 0.75 words
        - 128,000 tokens * 0.75 = 96,000 words
    - **Context Window: The memory capacity of the AI during a single conversation. It is measured in tokens (e.g., 128k tokens). It dictates how much text the AI can read and remember at one time before it starts forgetting the beginning of the chat.**
    - **Fine-Tuning: Taking an already trained model and giving it extra, specialized training on a narrow dataset (e.g., medical journals) to make it an expert in a specific topic.**
    - **Inference: The live process of the AI using its trained brain to calculate and generate an answer to your prompt.**
- **RAG ->**
    - Retrieval-Augmented Generation, an artificial intelligence technique that improves Large Language Model (LLM) responses by fetching relevant facts from an external database before generating an answer.

- It is a method to get better answers from GPT or any LLM model, by giving it relevant pieces of information along with the question.
- prevents hallucinations and provides accurate, up-to-date, or proprietary knowledge.
- RAG is done by:
  - taking a long text splitting it into pieces.
  - creating embeddings for each piece and storing those.
  - when someone asks a question, create an embedding for the question.
  - find the most similar embeddings of sections from the long text say the top 3.
  - send those 3 pieces along with the question to GPT for the answer.

- **Vanilla RAG ->**
  - Naive RAG, this system deals strictly with text.
  - **Data Handling: only ingests and retrieves text documents (like PDFs, text files, and FAQs).**
  - **Retrieval Process: Converts text into numerical vectors and matches the user's text query to the most relevant text chunks.**
  - **Use Case:** Best for answering questions from internal company wikis, text-based customer support, or conversational chatbots.
  - **Limitation: blind to visual or auditory data. If a PDF contains charts, diagrams, or images, traditional RAG ignores them, causing a loss of crucial context.**
- **Multimodal RAG ->**
  - **Data Handling: Processes and indexes a mix of text, images, and audio. For example, when reading a manual, it can understand a paragraph alongside the diagrams explaining it.**
  - **Retrieval Process: Maps different media types into a shared, multidimensional embedding space. This enables *any-to-any* retrieval meaning a user's text prompt can successfully retrieve an image or audio clip, and vice versa.**
  - **Use Case: Ideal for media-rich research, financial reports with complex charts, medical imaging diagnostics, or technical e-commerce applications.**

    - **Limitation: It is more complex and computationally heavier to implement, as it requires multimodal models to interpret visual cues.**
- If you are dealing with a particular use case, let's say, clinical research, try to find and use such a model which is mostly trained on Clinical data to get better results.
- **Vector DB ->**
    - stores data (text, images, etc.) alongside vector embeddings and provides fast similarity search (also called vector search or semantic search) at scale using vector indexes like HNSW(Hierarchical Navigable Small World) or ANN(Approximate Nearest Neighbors).
    - It allows users to find and retrieve similar objects quickly at scale in production.
    - Examples: Pinecone, Milvus, Weaviate, Qdrant, Chroma
- **Vector Embeddings ->**
    - numerical representation of data (like words, sentences, videos or images) that captures its meaning or properties. Example: [0.45, -0.21, 0.91, ...]
    - translates complex, unstructured human information into a long array of numbers so that machine learning models and AI can process and compare them mathematically.
- **Vector Search and Similarity Search ->**
    - Vector embeddings allow us to find and retrieve similar objects by searching for objects that are close to each other in the vector space. This concept is called ***vector search***, ***similarity search***, or ***semantic search*** database.
    - For example, just like we can find similar vectors for the word "Dog", we can find similar vectors to a search query. For example, to find words similar to the word "Kitten", we can generate a vector embedding for this query term - also called a query vector - and retrieve all its nearest neighbors, such as the word "Cat", as illustrated below.



- **Semantic Search ->**
    - refers to searching based on meaning and intent rather than exact keyword matches. Instead of asking “does this document contain the word cat?”, semantic search asks “is this document about cats?”—even if it never uses that exact word. Because vector embeddings capture contextual relationships (like dog, wolf, and cat being related animals), semantic search can return relevant results across synonyms, paraphrases, and even loosely related concepts.

- **Distance metrics used inside a Vector DB ->**
    - Distance metrics are the mathematical rulers that vector databases use to measure how similar two vectors are so that we can get accurate search results.
    - Usually, Vector DBs gives you options to choose Distance Metrics and you can set them according to your preference.
    - Some of the commonly used Distance Metrics are:
        - Squared Euclidean or L2-squared
        - Manhattan or L1
        - Cosine similarity
        - Dot product
        - Hamming distance
- **Vector Database vs. Traditional (Relational) Database ->**

- main difference between a modern vector and a traditional (relational) database comes from the type of data they were optimized for. While a relational database is designed to store structured data in columns, a vector database is also optimized to store unstructured data (e.g., text, images, or audio) and their vector embeddings.